URLs (`urls.dtx`)

Charles Duan

September 4, 2026

Every reference type can include a URL. *Hereinafter* formats URLs, makes them clickable links, and inserts line breaks into them, an important feature for law review articles where full URLs are included in paragraph-long citations. It also provides a number of alternate ways of displaying URLs, which may be more visually appropriate for different types of documents such as textbooks or legal memoranda.

To minimize dependencies on other packages, *Hereinafter* implements URL parsing, linking, and formatting on its own. It also provides user-level commands for handling URLs in text outside of citations. These features may be disabled with package options, as described below, to avoid conflicts with other packages.

1 Input Syntax

The url and opturl parameters add a URL to a reference. The URL should be entered as unformatted text, except that percent signs should be escaped with backslashes to avoid being interpreted as comment symbols.

URLs will allow line breaks after main punctuation, similar to the package `url.sty`.[1] If there is a long string of alphanumeric characters, then this package may not find a suitable breakpoint. Including `\␣` (that is, a backslashed space) in the URL will force a breakpoint to be inserted into the URL, with no effect on appearance.

`\HiUrl` {⟨*url-text*⟩}

`\url` This provides a user-level command for inserting a URL into text. At the beginning of the document, the command `\url` will be aliased to the same. If another package has already defined `\url` by then, a warning will be issued and *Hereinafter*'s definition will override it. This behavior may be altered by the package options overrideurl and nooverrideurl.

```
\newcommand\HiUrl[1]{%
    \hi@parseurl{#1}\reserved@a\reserved@a
}%
\AtBeginDocument{\hi@url@override}
\def\hi@url@override{%
    \@ifundefined{url}{}{%
```

[1] An exception is for the percent sign, where the line break will be inserted *before* the symbol, so that URL-encoded entities are kept intact.

```
        \PackageWarning\hi@pkgname{%
            Macro \string\url\space is already defined; overriding it%
        }%
    }%
    \let\url\HiUrl
}
```

`\hi@parseurl` {⟨*url*⟩}{⟨*control-sequence*⟩}

Parses ⟨*url*⟩ to produce the data structres described below, and defines ⟨*control-sequence*⟩ to content that will typeset the URL. The URL will have appropriate breakpoints put in, and can be safely inserted into a protected expansion environment such as `\protected@edef`.

The process is implemented in four steps. First, a unique number is generated to identify the URL. Second, the URL is parsed to construct a hyperlink href text. Third, the URL is parsed to construct a typeset text. Finally, the control sequence is defined.

```
\def\hi@parseurl#1#2{%
    \global\advance\hi@url@count\@ne
    \hi@url@link{#1}%
    \hi@url@text{#1}%
    \hi@url@assign{#2}%
}
\newcount\hi@url@count
```

`\hi@url@assign` {⟨*control-sequence*⟩}

This macro defines ⟨*control-sequence*⟩ to a typeset text for the URL most recently parsed. By default, the definition is the URL itself, but as described in `urls.dtx`, alternate forms may be used by redefining `\hi@url@assign`.

```
\def\hi@url@assign@default#1{%
    \letcs#1{hi@u\the\hi@url@count @text}%
    \@expand\if@endswithdot{\csname hi@u\the\hi@url@count @text\endcsname}{ii}%
        {\appto#1{\protect\@hi@dottrue}}%
}
\let\hi@url@assign\hi@url@assign@default
```

2 Data Structures

Every URL encountered will be assigned a number, which will be associated with data relating to the URL:

- `\hi@u#@link`: Text insertable as a PDF hyperlink declaration.
- `\hi@u#@text`: Text insertable into the body of a document, providing line breaking and spacing.
- `\hi@u#@declared`: Indicates that the URL has been declared for a PDF document.

3 Converting to PDF-Compatible Link

Parses a URL for purposes of forming it into a link URL. This primarily strips the URL of backslash escape characters. The result is placed in `\hi@u#@link`, where the number is `\hi@url@count`.

```
\def\hi@url@link#1{%
    % Force the link to start with https:// unless it has a protocol already
    \expandafter\protected@xdef\csname hi@u\the\hi@url@count @link\endcsname{%
        \find@try\find@start{%
            {http://}\@gobble
            {https://}\@gobble
            {ftp://}\@gobble
            {mailto:}\@gobble
        }{#1}{https://}%
        \hi@url@link@next{#1}%
    }%
}
\make@find@start{http://}
\make@find@start{https://}
\make@find@start{ftp://}
\make@find@start{mailto:}
\def\hi@url@link@next#1{%
    \find@word{#1}\hi@url@link@chars\hi@url@link@groups\hi@url@link@words{}%
}
\def\hi@url@link@words#1#2{%
    #1\hi@url@link@next{#2}%
```

```
}
\def\hi@url@link@groups#1#2{%
    \hi@url@link@next{\{#1\}#2}%
}
\def\hi@url@link@chars#1#2{%
    \@test \ifcat\relax\noexpand#1\fi{%
        % If the char is a macro
        \@ifundefined{hi@uchar@\string#1}{\string#1}{%
            \expandafter\@gobble\string#1%
        }%
    }{%
        % All other non-word chars. Ignore blank spaces.
        \ifblank{#1}{}{\detokenize{#1}}%
    }%
    \hi@url@link@next{#2}%
}
```

4 Inserting Link Code Into URLs

Once the URL has been converted into a link and saved, two things must occur for the link to be used:

1. The link must be "declared" the first time it is used in the PDF file.
2. Code invoking the now-declared link must be inserted surrounding typeset text.

The macros below perform these actions.

`\hi@url@declare` {⟨*num*⟩}

Inserts a `\special` into the document declaring the URL that has been assigned number ⟨*num*⟩. A macro `\hi@u`⟨*num*⟩`@link` should be defined as the properly escaped URL. This should be called once within the document.

```
\DeclareRobustCommand\hi@url@declare[1]{%
    \@ifundefined{hi@u#1@declared}{%
        \edef\reserved@a{%
            \noexpand\special{%
                pdf:object @HIURL#1 <<
                    /S /URI
                    /URI (\csname hi@u#1@link\endcsname)
                >>
            }%
        }\reserved@a
        \global\cslet{hi@u#1@declared}\@empty
    }{}%
}
```

`\hi@url@insert` {⟨*num*⟩}{⟨*text*⟩}

Inserts ⟨*text*⟩ with a PDF hyperlink to the url identified by ⟨*num*⟩.

```
\DeclareRobustCommand\hi@url@insert[2]{%
    \@ifundefined{hi@u#1@declared}{\hi@url@declare{#1}}{}%
    \noexpand\special{pdf:bann <<
        /Type /Annot
        /Subtype /Link
        /Border [0 0 0]
        /A @HIURL#1
    >>}#2\noexpand\special{pdf:eann}%
}%
```

`\hi@url@insertbox` {⟨*text*⟩}{⟨*num*⟩}

Inserts ⟨*text*⟩ within an `\hbox`, with a PDF hyperlink to the url identified by the current URL count number.

```
\DeclareRobustCommand\hi@url@insertbox[2]{%
    \hbox{\hi@url@insert{#2}{#1}}%
}
```

`\hi@url@box` {⟨*text*⟩}

In an expansion context, produces the tokens to be included in a macro defining the text of a URL inside a box.

```
\def\hi@url@box#1{%
    \noexpand\hi@url@insertbox{#1}{\the\hi@url@count}%
}
```

5 Converting to Text

Formats a URL as text intended for insertion into a paragraph. The resulting content is stored in the macro `\hi@u#@text`, where the number is `\hi@url@count`.

```
\def\hi@url@text#1{%
    \let\hi@url@acc\@empty
    \expandafter\protected@xdef\csname hi@u\the\hi@url@count @text\endcsname{%
        \leavevmode
        \hi@url@text@next{#1}%
    }%
}
```

The rest of these macros drop tokens within the expansion context of a `\protected@xdef`.

```
\def\hi@url@text@next#1{%
    \find@word{#1}{%
        \hi@url@text@chars
    }{%
        \hi@url@text@groups
    }{%
        \hi@url@text@words
    }{}%
}
```

#1 is the text to process; #2 is the rest of the URL.

```
\def\hi@url@text@words#1#2{%
    \hi@url@box{#1}%
    \hi@url@text@next{#2}%
}
```

#1 is a group; #2 is the rest of the URL. Insert the braces explicitly and continue processing.

```
\def\hi@url@text@groups#1#2{%
    \hi@url@text@next{\{#1\}#2}%
}
```

Parse a non-word character in a URL. #1 is the character, and #2 is the rest of the URL to be processed.

```
\def\hi@url@text@chars#1#2{%
    \@test \ifcat\relax\noexpand#1\fi{%
```

If the character (#1) is a command (e.g., `\%`), see if a definition is given for this command. If so, then execute the definition for the command (e.g., `\hi@uchar@\%`). Otherwise, treat the command like text in the URL and parse it along with the remainder of the URL.

```
        \@ifundefined{hi@uchar@\string#1}{%
            \expandafter\hi@url@text@next\expandafter{\string#1#2}%
        }{%
            \csname hi@uchar@\string#1\endcsname{#2}%
        }%
    }{%
```

If #1 is not a command, then see if a definition is given for this character. If so, execute that definition. Otherwise, just add the character to the URL. In all cases, continue with parsing the remainder of the string.

```
        \@ifundefined{hi@uchar@\@backslashchar\string#1}{%
            \hi@url@box{#2}%
            \hi@url@text@next{#2}%
        }{%
            \csname hi@uchar@\@backslashchar\string#1\endcsname{#2}%
        }%
    }%
}
```

We now define macros of the form `\hi@uchar@`⟨*char*⟩ for each character that will receive special processing. Each macro will take one argument: `#1` the trailing text of the URL. The macro is responsible for continuing URL processing with `\hi@url@text@next`.

The original lists of characters were borrowed from url.sty, by Donald Arsenau. At this point the code no longer resembles the original beyond standard TeX programming conventions, but I'd like to credit the original package.

These characters accept a breakpoint after them, so long as the next characters in the URL are word characters.

```
\def\do#1{%
    \@namedef{hi@uchar@\string #1}##1{%
        \hi@url@box{\expandafter\@gobble\string#1}%
        \@ifstartswithwordchar{##1}{%
```

```
            \noexpand\hi@uchar@break
        }{}%
        \hi@url@text@next{##1}%
    }%
}
\do\.\do\@\do\!\do\;\do\]\do\)\do\,\do\+\do\=\do\#\do\:
```

These characters accept a breakpoint *and* add a little extra space.

```
\def\do#1{%
    \@namedef{hi@uchar@\string #1}##1{%
        \noexpand\hi@uchar@spacenobreak
        \hi@url@box{\expandafter\@gobble\string #1}%
        \find@word{##1}%
            {\@gobbletwo}%
            {\noexpand\hi@uchar@spacebreak\@gobbletwo}%
            {\noexpand\hi@uchar@spacebreak\@gobbletwo}%
            {}%
        \hi@url@text@next{##1}%
    }%
}
\do\/\do\?\do\&\do\-
```

These characters accept no breakpoints but are often backslashed; including them forces the removal of the backslash.

```
\def\do#1{%
    \@namedef{hi@uchar@\string #1}##1{%
        \hi@url@box{\expandafter\@gobble\string #1}%
        \hi@url@text@next{##1}%
    }%
}
\do\^\do\$
```

These characters require special treatment when they are seen, and a breakpoint is inserted if the next character is a word character.

```
\def\do#1#2{%
    \@namedef{hi@uchar@\string #1}##1{%
        \hi@url@box{\unexpanded{#2}}%
        \@ifstartswithwordchar{##1}{%
            \noexpand\hi@uchar@break
        }{}%
        \hi@url@text@next{##1}%
    }%
}
\do\<{$\protect\langle$}%
% There's some bug in doc.sty that requires this
\expandafter\do\csname|\endcsname{$\protect\mid$}%
\do\{{$\protect\lbrace$}%
\do\\{$\protect\backslash$}%
\do\_{\_}%
```

These characters require special treatment and are not legal breakpoints.

```
\def\do#1#2{%
    \@namedef{hi@uchar@\string #1}##1{%
        \hi@url@box{\unexpanded{#2}}%
        \hi@url@text@next{##1}%
    }%
}
\do\>{$\protect\rangle$}
\do\}{$\protect\rbrace$}
\do\~{\char`\~}
```

A space indicates to insert a break alone.

```
\@namedef{hi@uchar@\string\ }#1{%
    \noexpand\hi@uchar@break
    \hi@url@text@next{#1}%
}
\@namedef{hi@uchar@\string\^^M}#1{%
    \noexpand\hi@uchar@break
    \hi@url@text@next{#1}%
}
```

The percent sign indicates a URL-encoded value. It consumes two tokens from the trailing text and puts a breakpoint before the percent sign and after the two tokens. (No check is made to determine what those tokens are).

```
\@namedef{hi@uchar@\string\%}#1{%
    \noexpand\hi@uchar@spacebreak
    \ifstrempty{#1}{%
```

```
        \hi@url@box{\noexpand\%}%
        \hi@url@text@next{}%
    }{%
        \hi@url@pct#1\@stop
    }%
}%
\def\hi@url@pct#1#2\@stop{%
    \ifstrempty{#2}{%
        \hi@url@box{\noexpand\%\detokenize{#1}}%
        \hi@url@text@next{}%
    }{%
        \hi@url@pct@{#1}#2\@stop
    }%
}
\def\hi@url@pct@#1#2#3\@stop{%
    \hi@url@box{\noexpand\%\detokenize{#1#2}}%
    \noexpand\hi@uchar@spacebreak
    \hi@url@text@next{#3}%
}
```

Macros for types of breakpoints and spaces.

```
\newskip\urlbreakskip \urlbreakskip=0pt plus .4pt
\DeclareRobustCommand\hi@uchar@break{%
    \penalty\exhyphenpenalty
}
\DeclareRobustCommand\hi@uchar@spacenobreak{%
    \unskip\unpenalty\penalty\@M
    \hskip\urlbreakskip
}
\DeclareRobustCommand\hi@uchar@spacebreak{%
    \penalty\exhyphenpenalty\hskip\urlbreakskip
}
```

6 Alternate Presentations of URLs

By default, URLs are shown in text. This format, logical for print publications with no other easy access to hyperlinks, can look lengthy and cumbersome in legal memoranda and other contexts. Accordingly, several other options are provided.

In documents with tables of authorities, URLs can be presented in an alternate format: The URLs are included only in the table listing, and citations in the document include a marker "*available online*." A footnote is added after the first such omitted URL, informing the reader that omitted URLs are in the Table of Authorities. To select this format, use the package option toaurl.

This format has the advantage of producing cleaner-looking briefs, particularly given the large font-to-text-field ratios that most courts require of briefs (such that URLs could take up a large fraction of a page). The main disadvantage is that it actually increases word count, since URLs typically count only as one word.

The text to be shown in place of URLs can be redefined with the macro `\ToaUrlMark`. The text to be shown in the footnote after the first such replaced URL can be redefined with the macro `\ToaUrlText`.

`\ToaUrlMark`
`\ToaUrlText`

`\hi@showurl` All displays of URLs are filtered through `\hi@showurl{`⟨*text*⟩`}`. By default, it just displays its argument.

```
\def\hi@urlstyle@full{%
    \def\hi@showurl##1{##1}%
}
```

Provides for URL being shown only in the table of authorities. #1 is the pre-URL signal; #2 is the URL itself.

```
\def\hi@urlstyle@toa{%
    \def\hi@showurl##1{%
        \if@hi@in@toa
            ##1%
        \else
            \ToaUrlMark
            \hi@showurl@toatext
        \fi
    }
    \def\hi@showurl@toatext{%
        \hi@defer@content{\ToaUrlText}%
        \global\let\hi@showurl@toatext\@empty
    }
    \def\ToaUrlText{%
        Locations of authorities available online are shown in the Table of
        Authorities.
    }
    \def\ToaUrlMark{\emph{available online}}
}
```

Alternatively, in documents intended to be used only electronically, URLs can be replaced with a word that is hyperlinked. To select this format, use the package option linkurl.

`\LinkText` The text to be used in place of the URL is defined in the macro `\LinkText`. No line break protection is performed on the word, so it should be an unbreakable word or the macro should be defined to include an unbreakable `\hbox`.

```
\def\hi@url@assign@link#1{%
    \edef#1{%
        \noexpand\leavevmode
        \noexpand\hi@url@insert{\the\hi@url@count}{\noexpand\LinkText}%
        \noexpand\hi@linkurl@auxwrite{\the\hi@url@count}%
    }%
}
\def\hi@urlstyle@link{%
    \let\hi@url@assign\hi@url@assign@link
}
\def\hi@linkurl@auxwrite#1{%
    \addtocontents{tou}{\protect\l@hi@tou{#1}{\thepage}}%
}
\def\LinkText{\emph{link}}
```

If the URL link style is selected, then a table of URLs may be generated with the command `\TableOfUrls`. This ensures that print copies of a document contain any URLs that were otherwise omitted by replacement with the `\LinkText` word.

```
\def\TableOfUrls{%
    \makeatletter
    \@input{\jobname.tou}%
    \makeatother
    \if@filesw
        \newwrite\tf@tou
        \immediate\openout\tf@tou \jobname.tou\relax
    \fi
}
\def\l@hi@tou#1#2{%
    \@dottedtocline{1}{0em}{1.5em}{\csname hi@u#1@text\endcsname}{#2}%
}
```